

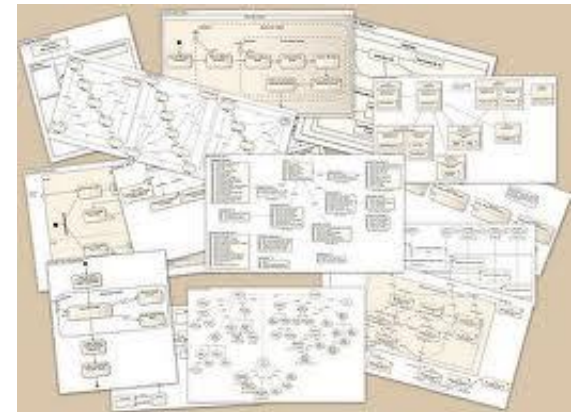
UML 2.0: INTRODUZIONE

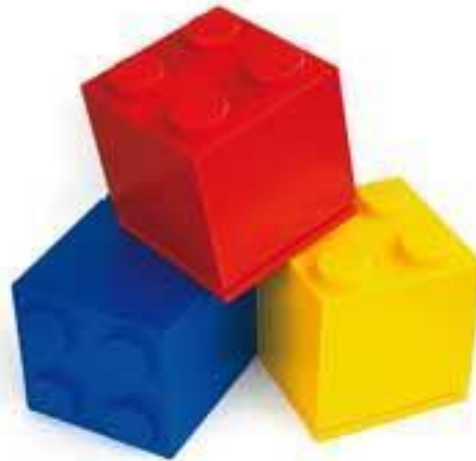
Ingegneria del Software a.a. 2025-26

AGENDA



- Che cosa è UML?
 - Cosa non è UML ...
- Come usare UML?
- Prospettiva Concettuale vs. Software
- Un po di storia ...
- Perché è importante conoscere UML?
- Notazioni e meta-modello
- I 14 diagrammi (UML 2.5)
 - Ne vedremo solo alcuni
- Come integrare UML nel processo?





Martin Fowler

UML DISTILLED

Guida rapida al linguaggio
di modellazione standard

quarta edizione

Edizione italiana a cura di Luciano Baresi



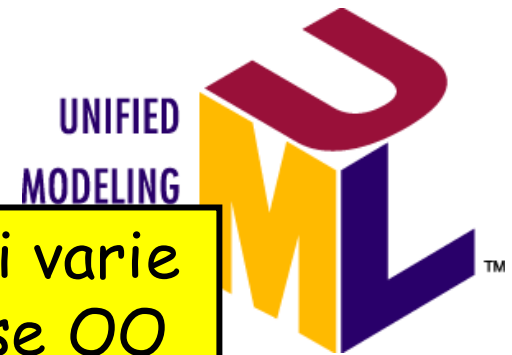
CHE COS'È UML?

- UML = Unified Modeling Language
- Famiglia di **notazioni grafiche** utili a supportare la descrizione e il progetto dei sistemi software
 - In particolare sistemi OO
 - Ma anche altro:
 - Sistemi hardware
 - Dominio del business
 - ...
- **Standard OMG**
 - Object Management Group
 - Consorzio di aziende IT
 - Not-for-profit
 - Obiettivo: creare standard nel contesto IT



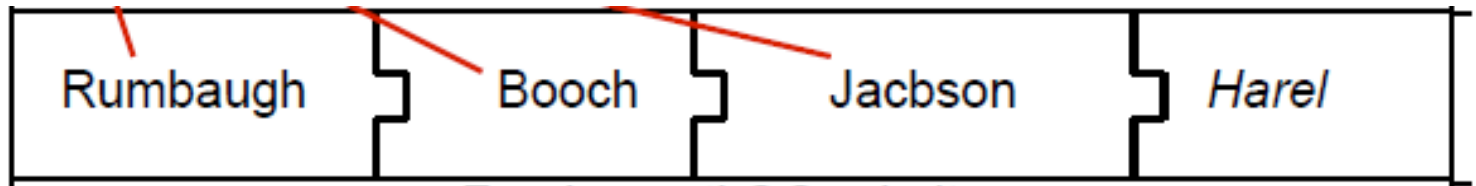
CHE COS'È UML?

- UML = Unified Modeling Language
- Famiglia di **notazioni grafiche** utili a supportare la descrizione e il



Unificato = in pratica una collezione di varie notazioni preesistenti, integrate e rese OO

- Ma anche altro:



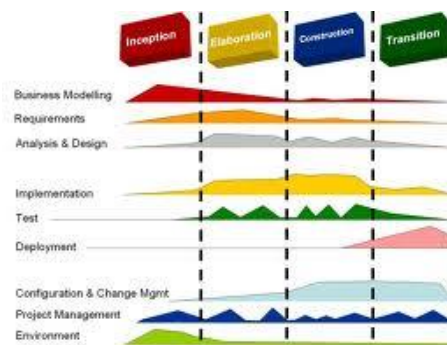
○ Standard OMG

- Object Management Group
 - Consorzio di aziende IT
 - Not-for-profit
 - Obiettivo: creare standard nel contesto IT



COSA NON È UML

- **Non è un metodo di sviluppo (metodologia)!**
 - È un linguaggio/notazione
- **Non è un linguaggio di programmazione**
 - È un linguaggio di modellazione
- **Non è legato a UP**
 - Non è legato a nessun processo particolare
 - Può essere usato sia nei metodi plan-driven che in quelli agili



No!

I MOLTI MODI DI USARE UML

○ **Abbozzo (sketch)**

- **Scopo:** aiutare la comunicazione e la discussione
 - Solo info + importanti (diagrammi incompleti)
- **Tool:** editor grafici o lavagna

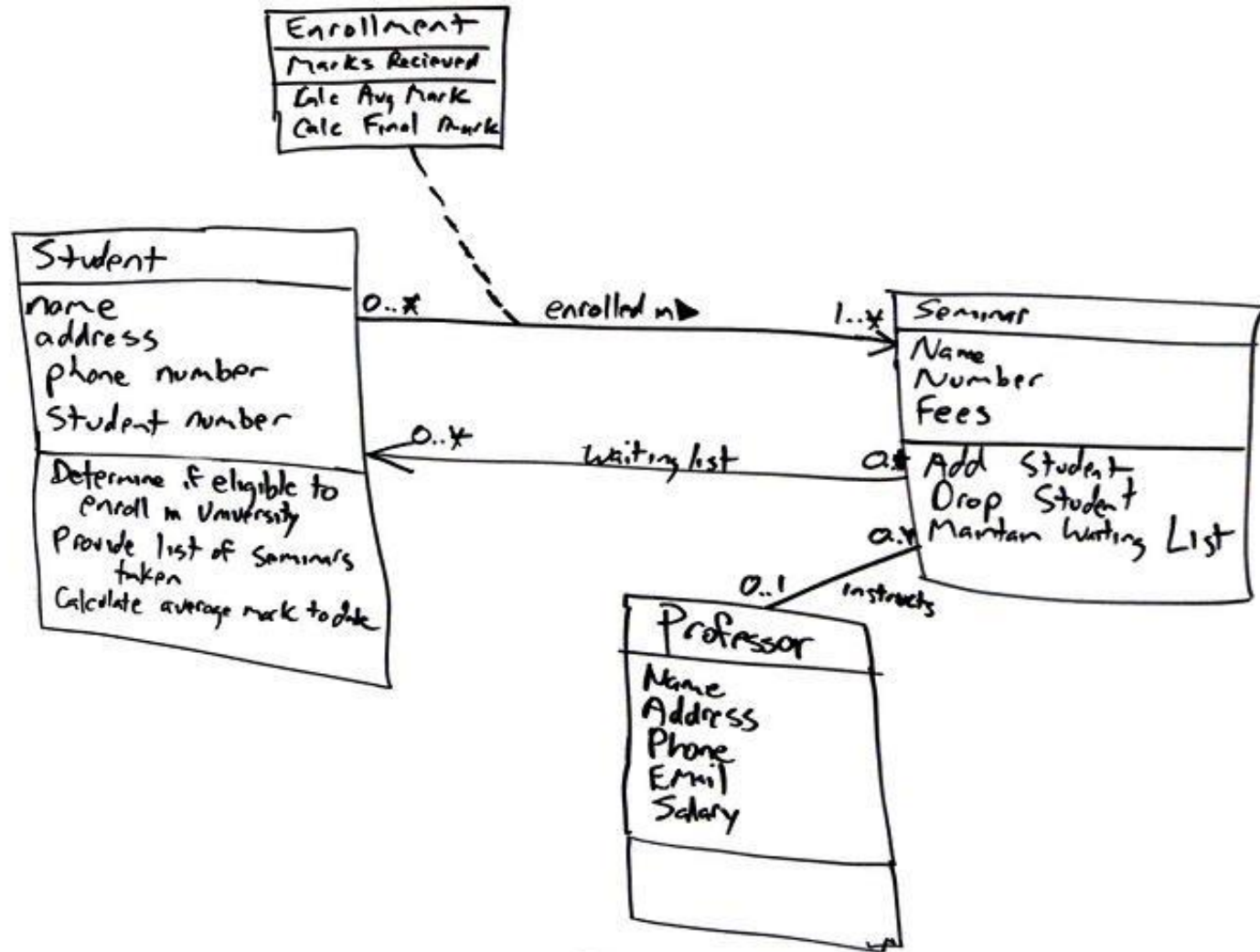
○ **Progetto dettagliato (blueprint)**

- **Scopo:** fornire un modello completo/dettagliato da implementare
 - Codifica = attività meccanica
- **Tool:** UML modeler
 - Es. Visual Paradigm, Papyrus

○ **Linguaggio di programmazione**

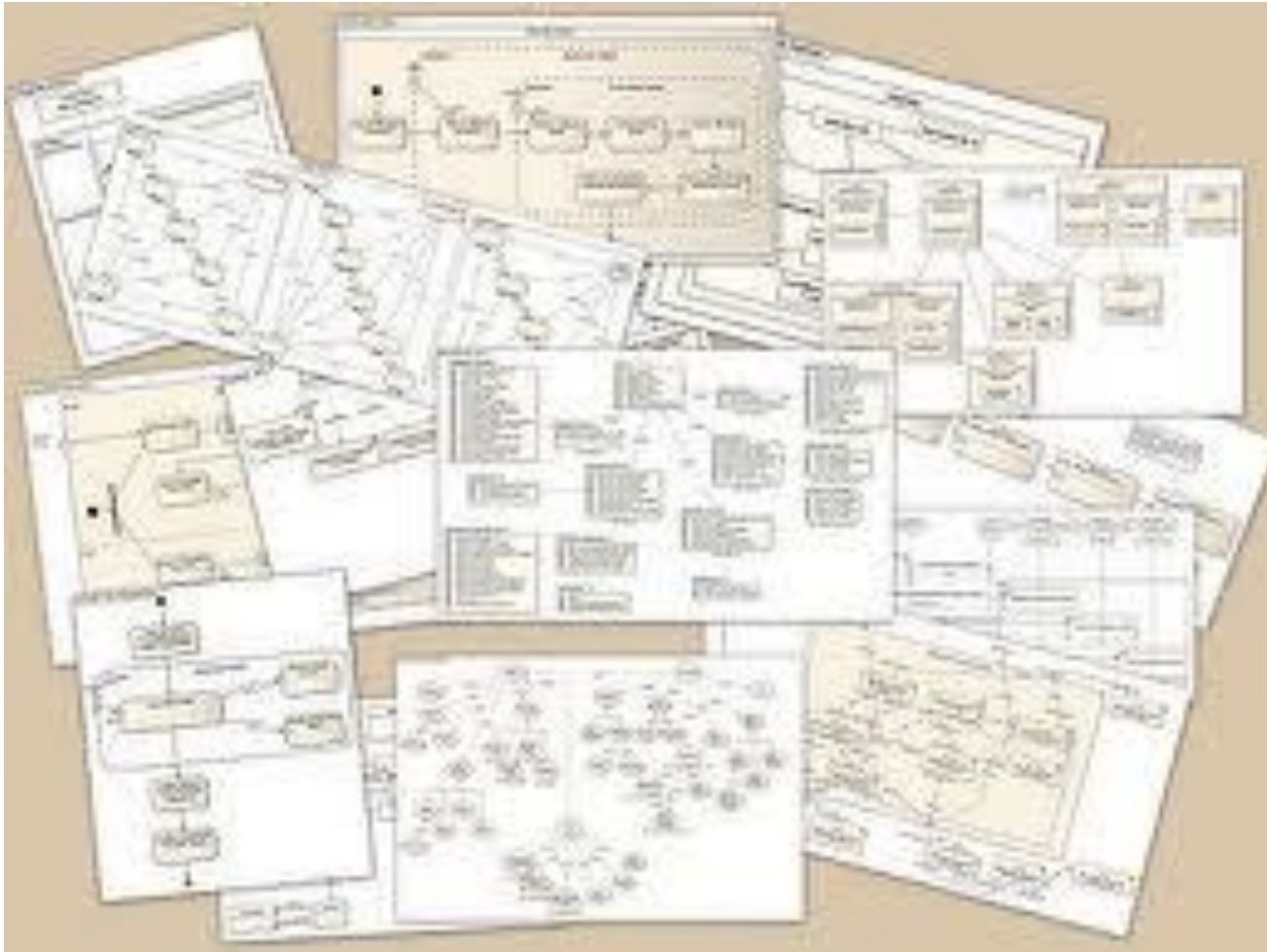
- **Scopo:** fornire un “modello eseguibile”
- **Tool:** specifici per eseguire/trasformare il modello
 - Es. UniMod / WebRatio (IFML), trasformazioni ad-hoc

ESEMPIO: ABBOZZO

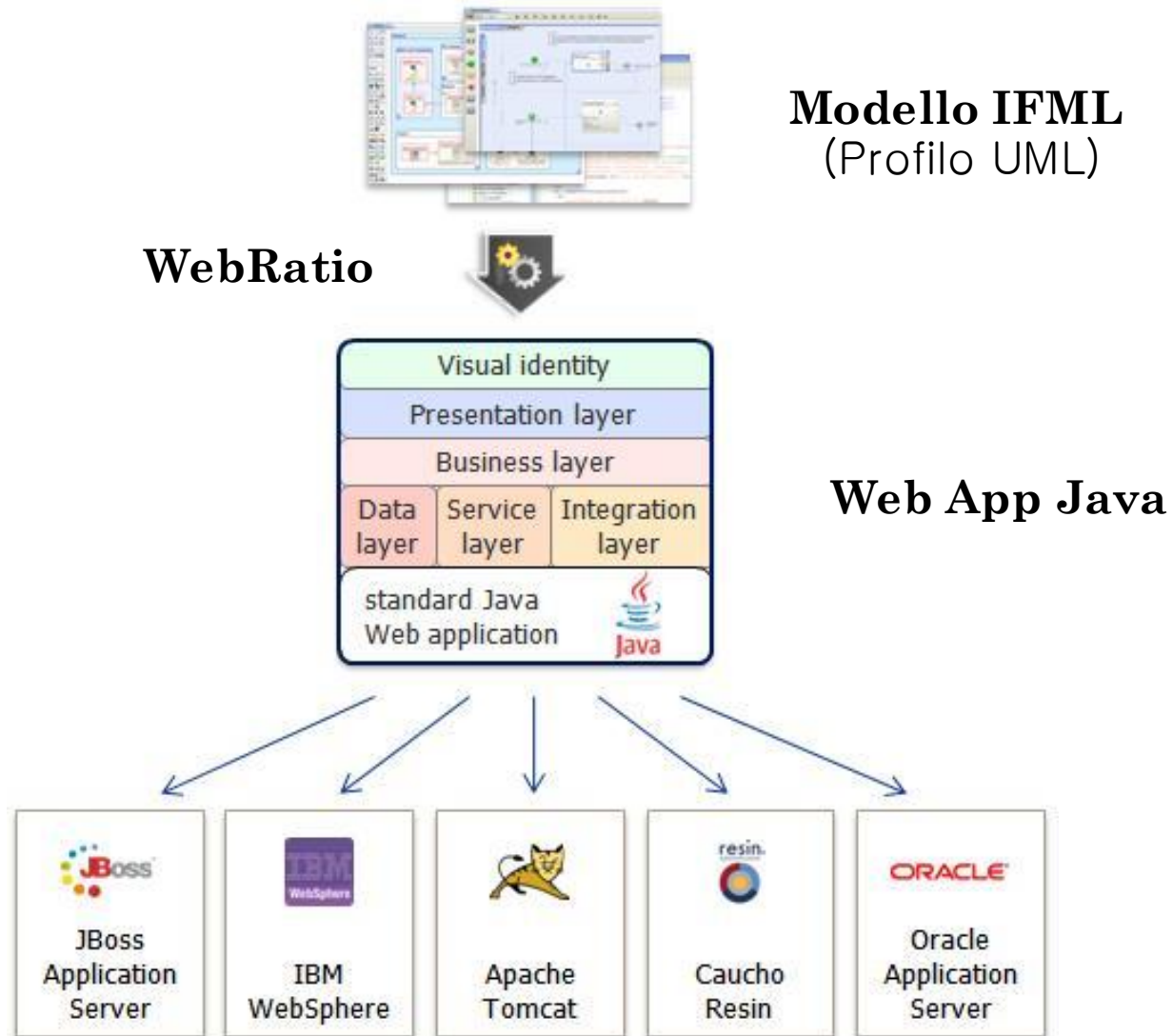


ESEMPIO: DESIGN DETTAGLIATO

Visual Paradigm



ESEMPIO: LINGUAGGIO DI PROGRAMMAZIONE



○ Prospettiva software

- Elementi rappresentati sono *elementi di un sistema software*
 - Es. Utente: è la classe in un linguaggio di programmazione, ad esempio con gli attributi nome, cognome e indirizzo
- Serve per **descrivere il design di un sistema software**
- Serve per documentare un sistema software

○ Prospettiva concettuale

- Elementi rappresentati sono *concetti del dominio*
 - Es. Utente: è la persona fisica
- Serve per modellare il **dominio del business e i processi**

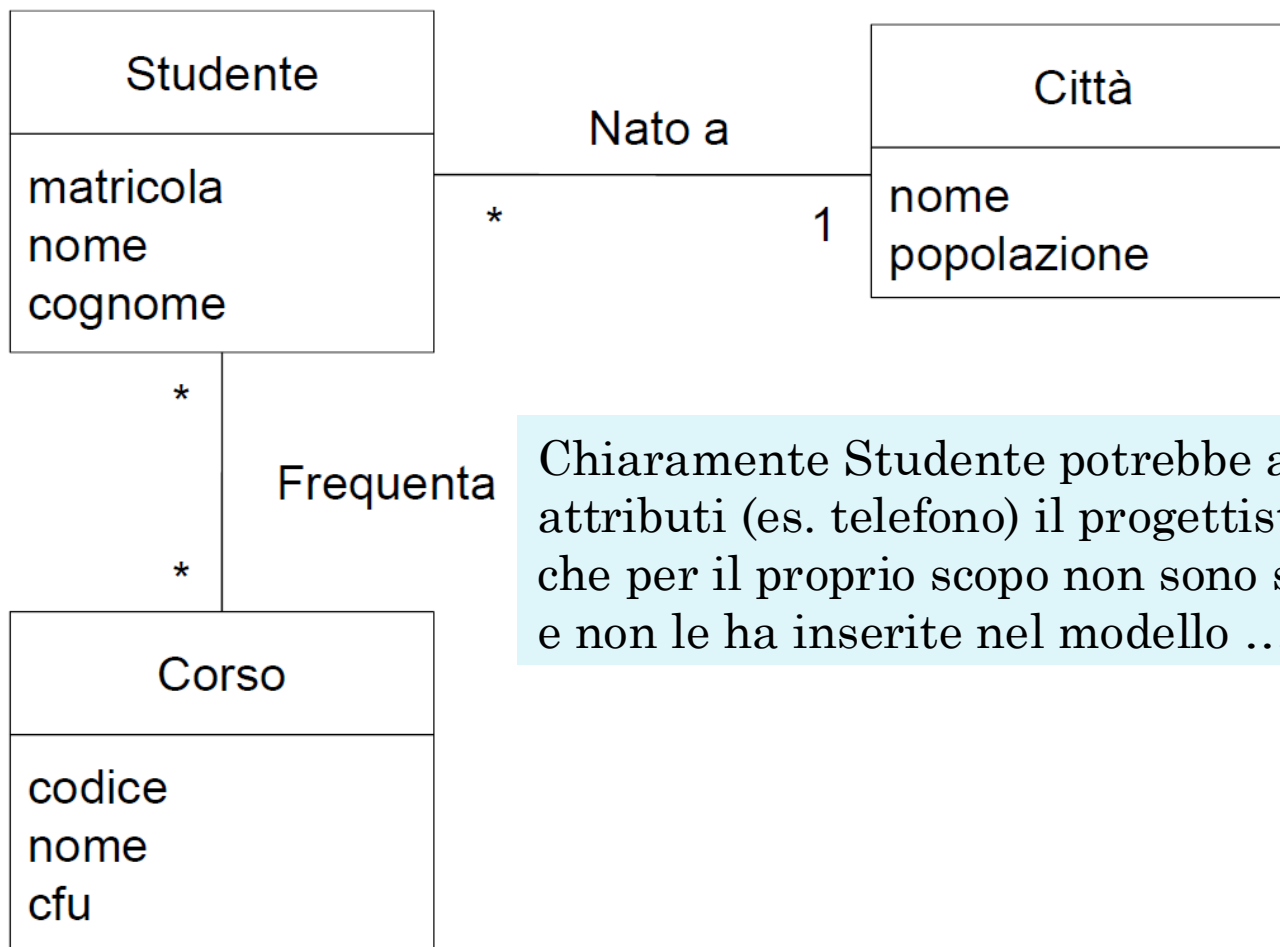
Spesso nello sviluppo di un software il primo passo è modellare il dominio del business ...

MODELLO DI DOMINIO (DEL BUSINESS)

- E' una rappresentazione *visuale* di classi **concettuali** o di oggetti del mondo reale di un dominio [Fowler]
- E' un **glossario visuale** delle **astrazioni significative** del dominio
 - Non si modella tutto! Solo i concetti più importanti e rilevanti nel contesto
- Utile:
 - Per chiarire i concetti di dominio nel quale il sistema software che dobbiamo progettare andrà inserito
 - Glossario comune tra i vari stakeholder
 - Per ispirare nomi di alcune classi e attributi software nel design
 - Per la progettazione del sistema
 - Punto di partenza

ESEMPIO DI MODELLO DI DOMINIO

Rettangoli rappresentano classi concettuali

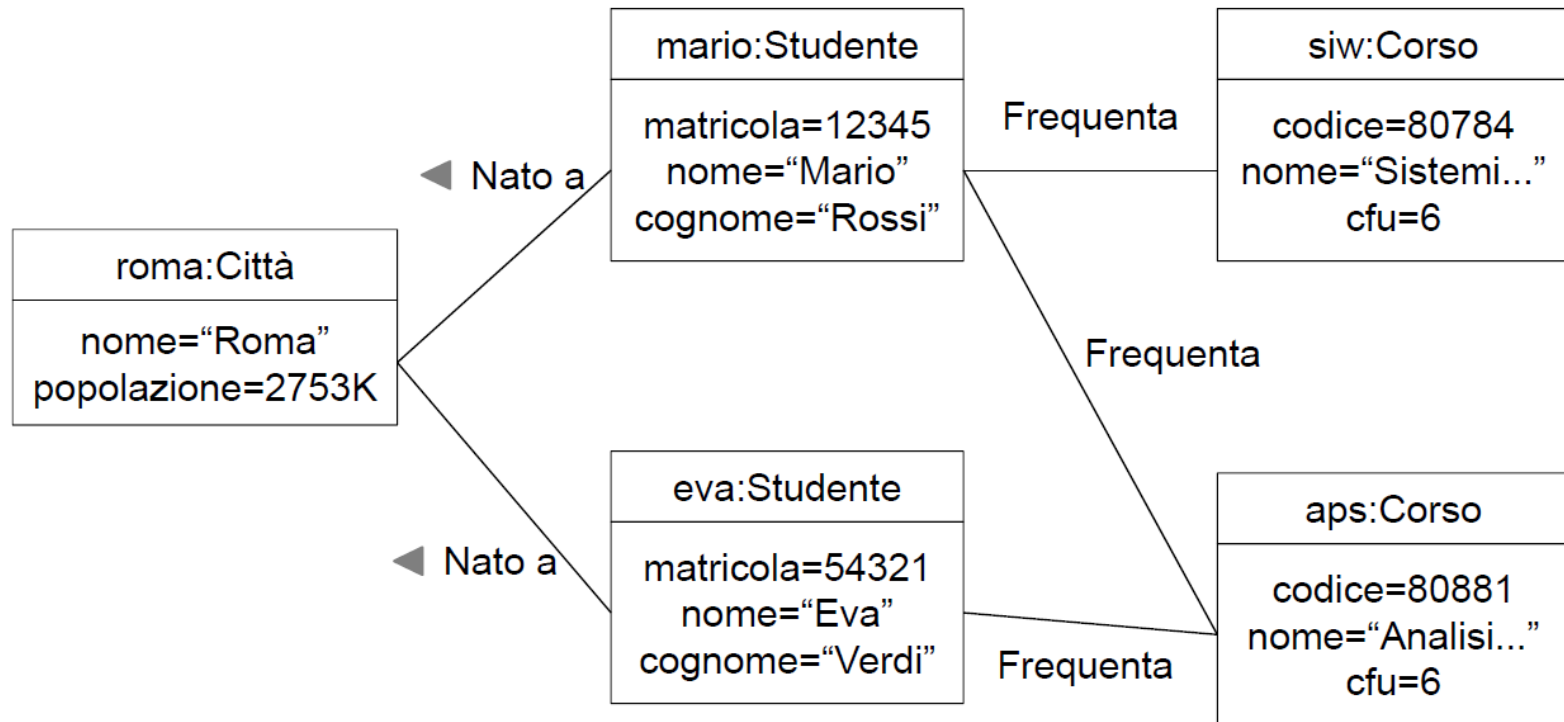


Chiaramente Studente potrebbe avere più attributi (es. telefono) il progettista ha deciso che per il proprio scopo non sono significative e non le ha inserite nel modello ...

OGGETTI DI DOMINIO

Oggetti di dominio per rappresentare

- uno studente Mario Rossi, nato a Roma, frequenta SIW e APS
- una studentessa Eva Verdi, nata a Roma, frequenta APS



UN PO' DI STORIA ...

'80 C++ small talk

Peter Coad

Jim Rumbaugh



OMG

Sally Shlaer
Steve Mellor

Ivar Jacobson



Mary Loomis

Unified Modelling
Language 1.0

Rebecca Wirfs-Brock

UML 1.1

UML 1.2

UML 1.3

UML 1.4

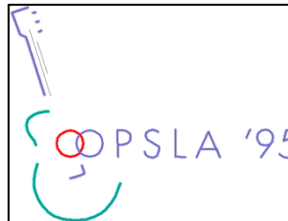
UML 1.5

Jim Odell

Grady Booch

IBM

Rational



Unified Method 0.8



PERCHÈ STUDIARE UML?

- È uno standard
 - UML svolge un'importantissima funzione di “**lingua franca**” nel mondo ICT
- Abbastanza usato/richiesto dall'industria
 - Almeno a livello di abbozzo
 - Come blueprint è richiesto in alcuni appalti pubblici
- Un immagine (diagramma) vale 1000 parole!
- Indispensabile per le figure di **Analista** e **Progettista** (architetto) software

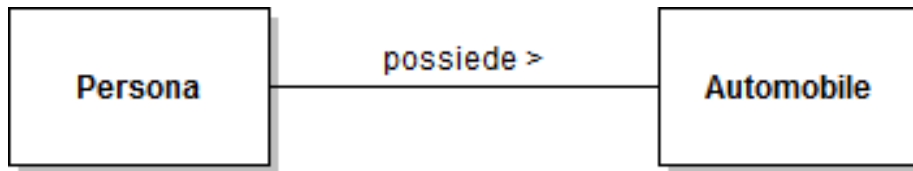
PERCHÈ NON STUDIARLO IN MODO APPROFONDITO?

- Complesso per un corso come questo
- Approccio **blueprint** poco usato nell'industria
 - Complesso anche per i progettisti + bravi proporre un design completo che sia privo di errori
 - Difficile perché il modello non si può testare/eseguire
- Non esistono studi empirici che provano che l'approccio blueprint è il migliore
- Mio punto di vista uguale a quello di M. Fowler

"I progetti dettagliati sono troppo complessi da realizzare e rallentano il processo di sviluppo"

NOTAZIONE E META-MODELLO

- UML definisce una (famiglia di) notazione(i) e un meta-modello
- **Notazione = sintassi grafica del linguaggio**
 - Es. la notazione dei diagrammi delle classi definisce come rappresentare classi e associazioni

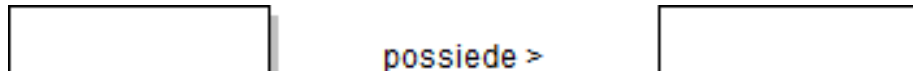


- **Meta-modello = definisce i concetti stessi del linguaggio**
 - Es. Cosa è una Classe, un Attributo, un Operazioni
- Quale è l'impatto del meta-modello sull'uso della notazione?

Poco o nulla se usiamo UML come abbozzo

NOTAZIONE E META-MODELLO

- UML definisce una (famiglia di) notazione(i) e un meta-modello
- **Notazione = sintassi grafica del linguaggio**
 - Es. la notazione dei diagrammi delle classi definisce come rappresentare classi e associazioni

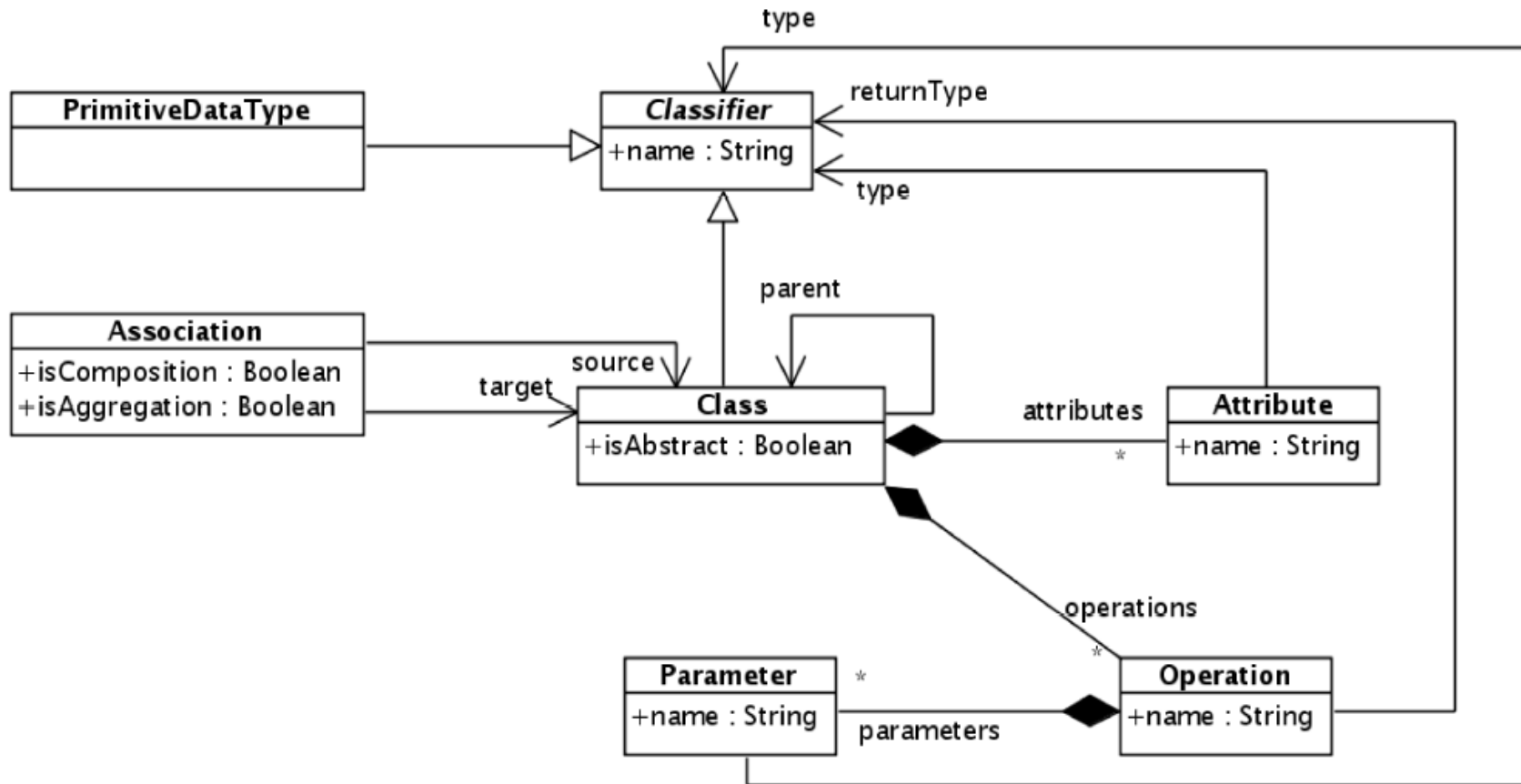


Possiamo vivere anche senza meta-modello!!!

- **Meta-modello = definisce i concetti stessi del linguaggio**
 - Es. Cosa è una Classe, un Attributo, un Operazioni
- Quale è l'impatto del meta-modello sull'uso della notazione?

Poco o nulla se usiamo UML come abbozzo

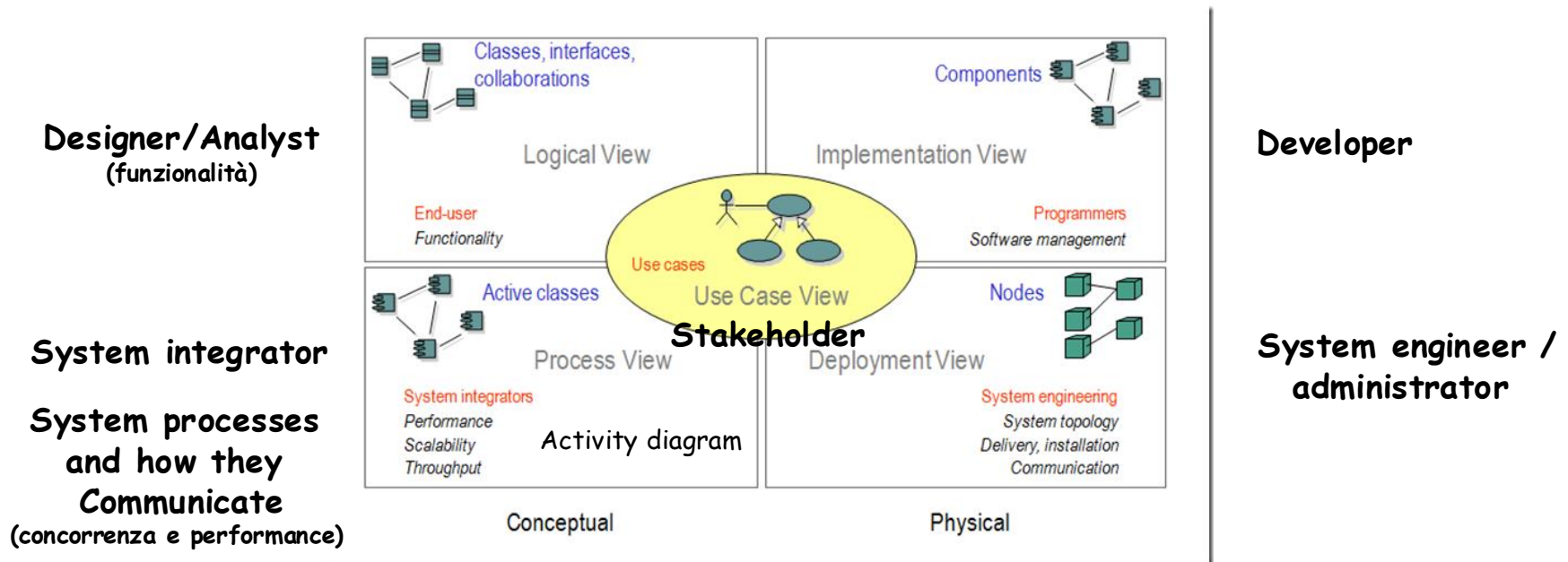
ESEMPIO DI META-MODELLO



Espresso a sua volta in UML

VISTE, DIAGRAMMI E MODELLO (IN UML)

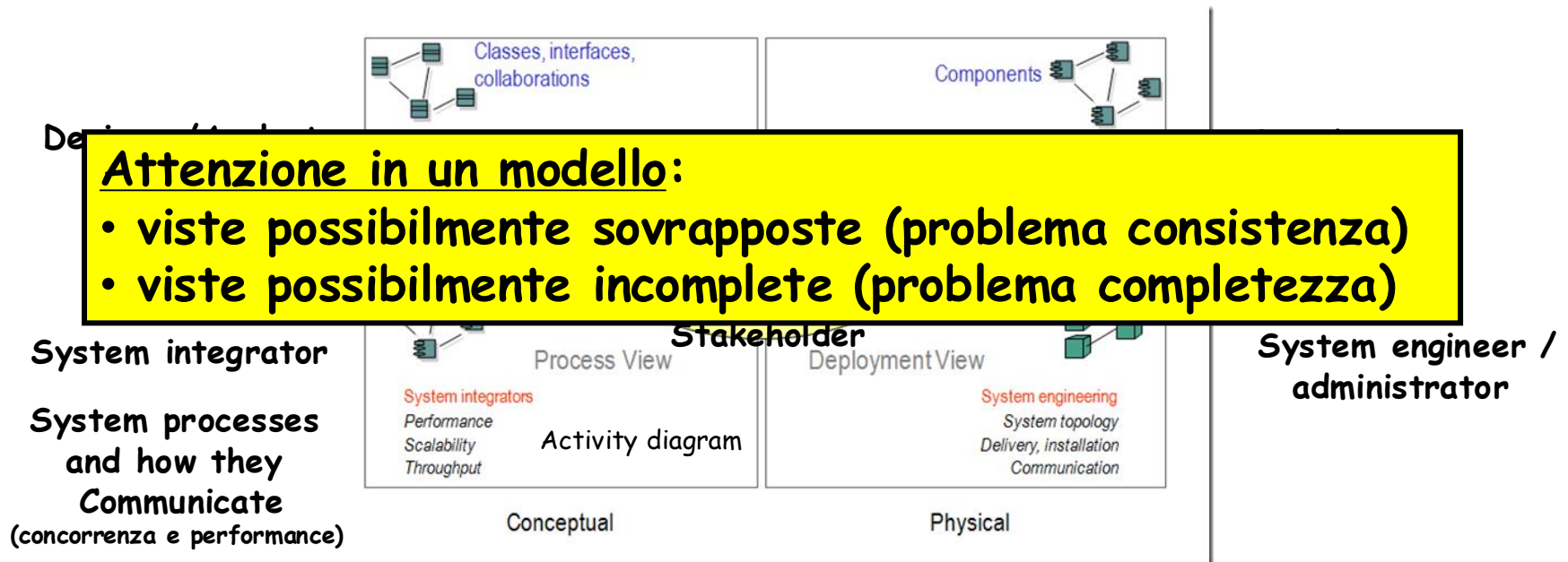
- In UML, un sistema viene descritto utilizzando diverse **viste**:
 - Vista = “*particolare aspetto di un Sistema dal punto di vista di uno specifico ruolo*”



- Un diagramma descrive/presenta una **view/subview**
- Chiameremo **Modello** un insieme di diagrammi che descrivono aspetti diversi dello stesso sistema

VISTE, DIAGRAMMI E MODELLO (IN UML)

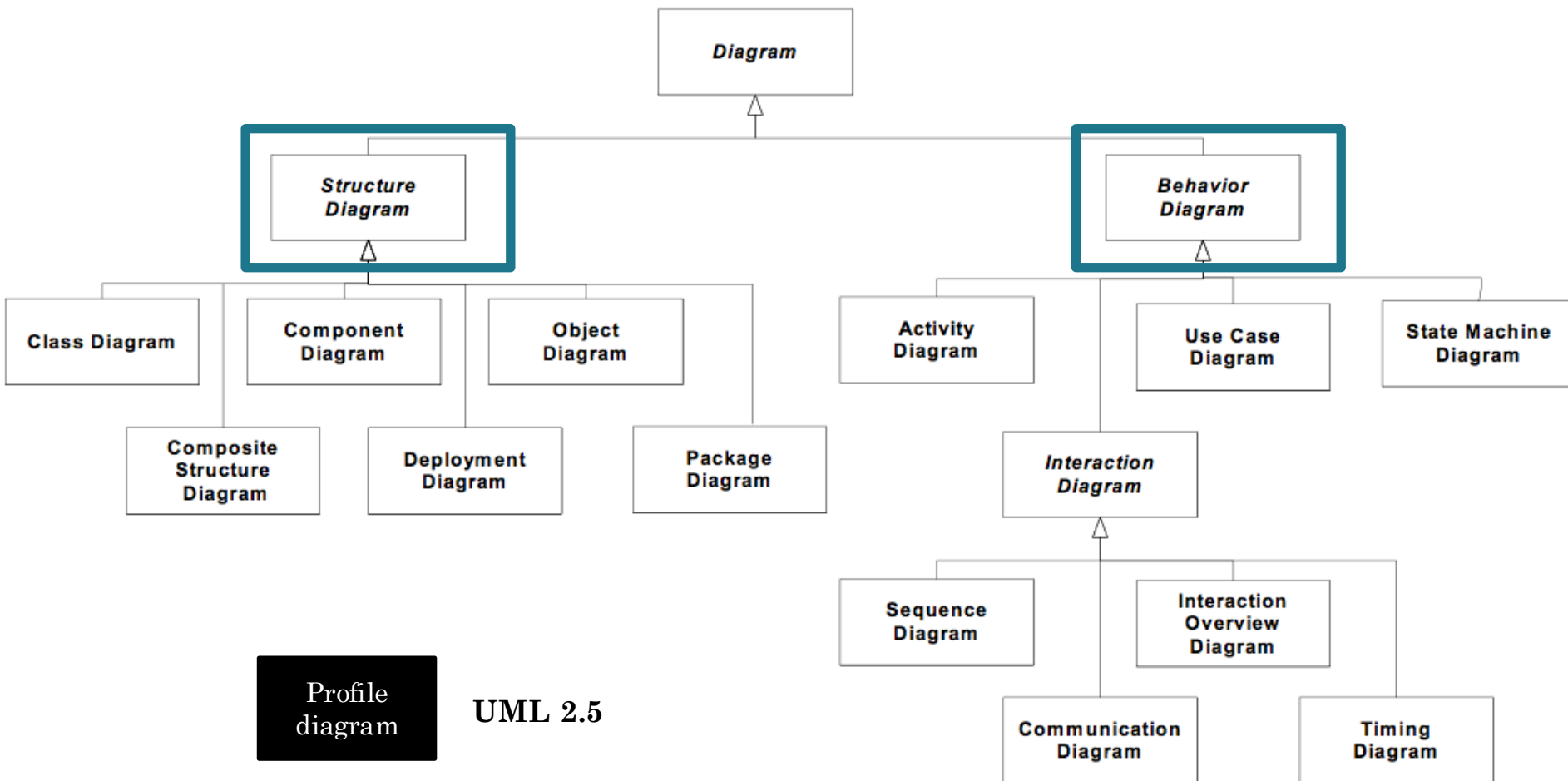
- In UML, un sistema viene descritto utilizzando diverse **viste**:
 - Vista = “*particolare aspetto di un Sistema dal punto di vista di uno specifico ruolo*”



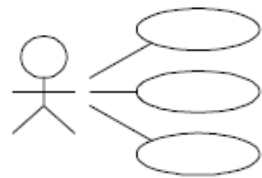
- Un diagramma descrive/presenta **una view/subview**
- Chiameremo **Modello** un insieme di diagrammi che descrivono aspetti diversi dello stesso sistema

DIAGRAMMI UML

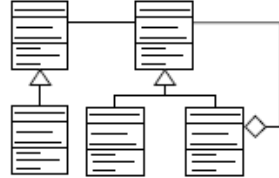
UML 2.0 comprende 13 tipi di diagrammi (nel 2.5 sono 14)



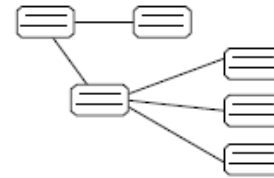
QUELLI CHE VEDREMO



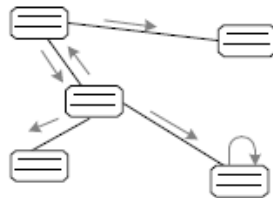
Use Case
(Casi D'uso)



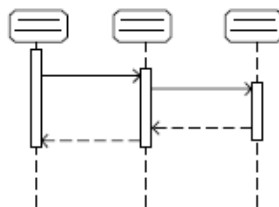
Class Diagram
(Diagrammi delle Classi)



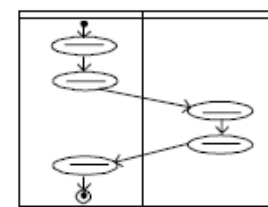
Object Diagram
(Diagrammi degli Oggetti)



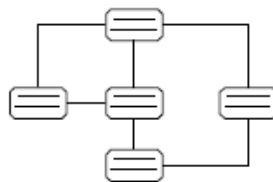
Communication Diagram
(Diagramma di comunicazione)



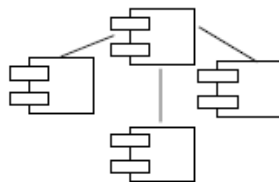
Sequence Diagram
(Diagramma di Sequenza)



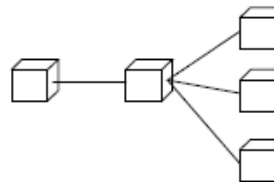
Activity Diagram
(Diagramma di Attivita')



State machine Diagram
(dmacchina a stati)

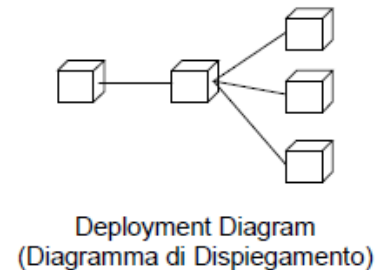
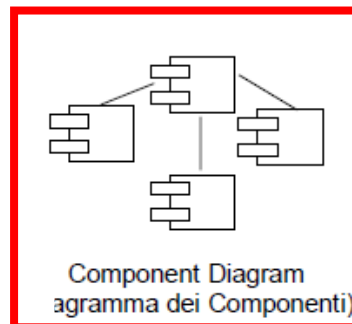
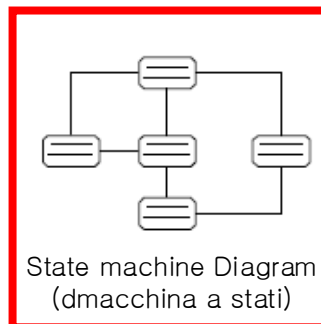
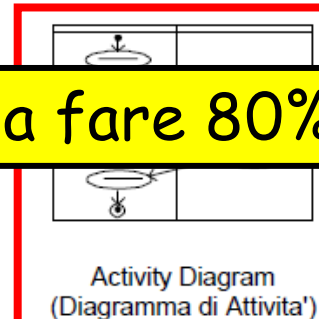
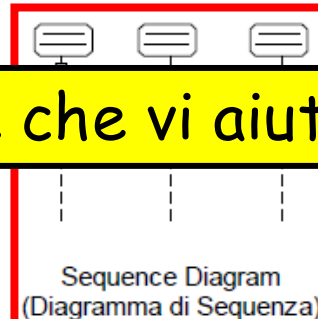
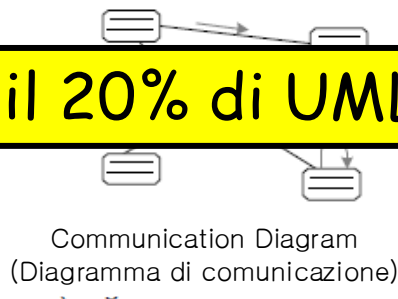
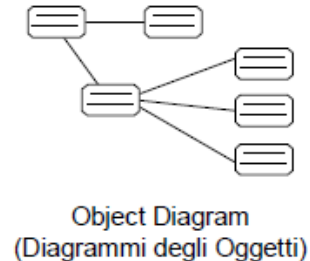
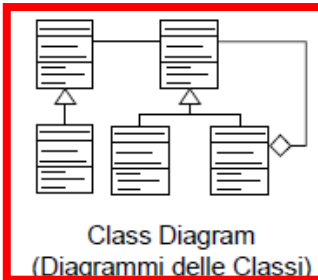
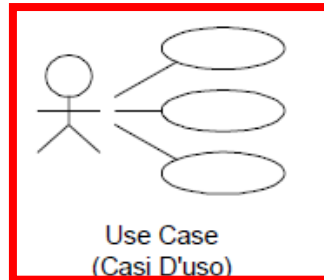


Component Diagram
agramma dei Componenti)



Deployment Diagram
(Diagramma di Dispiegamento)

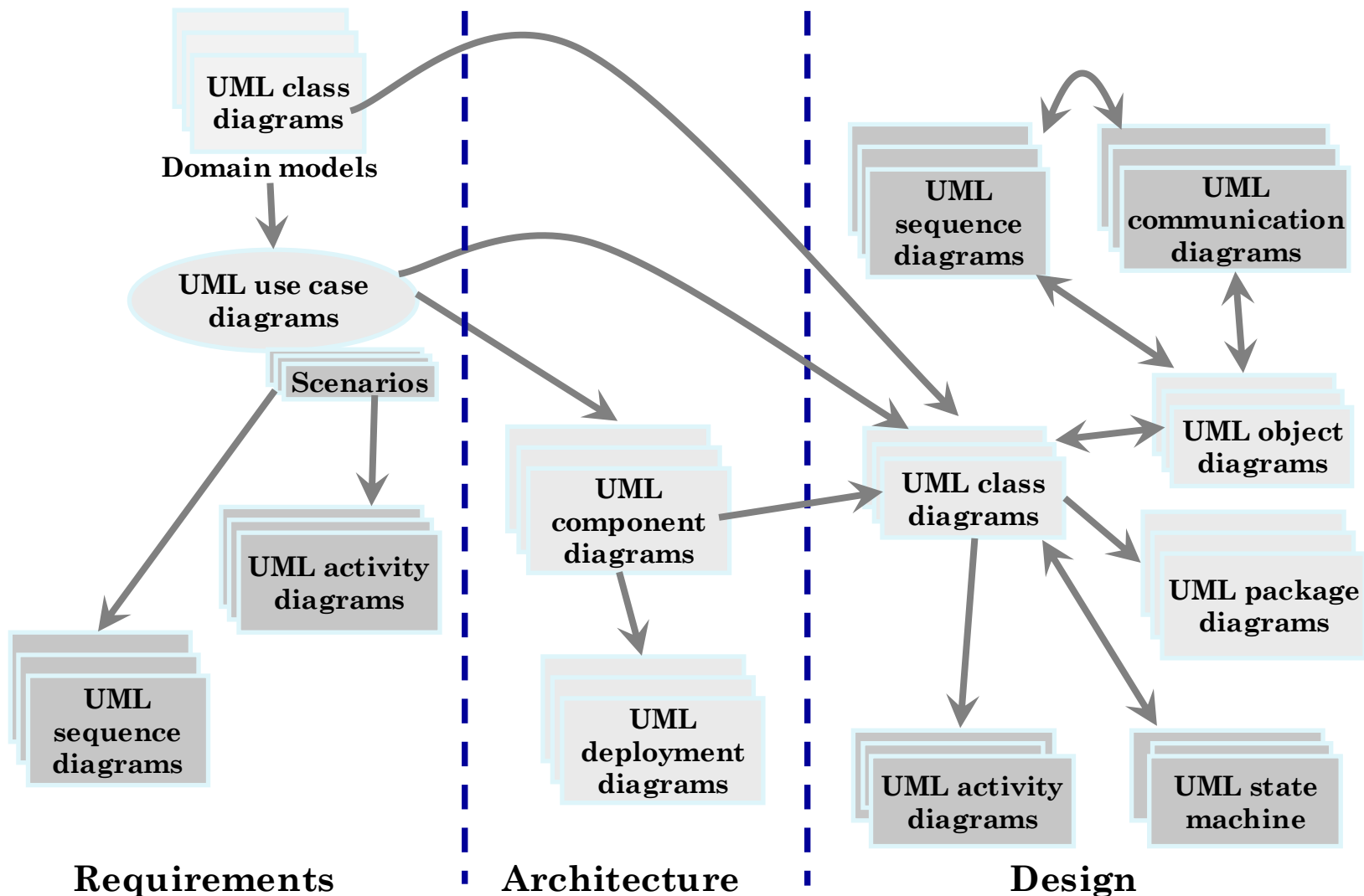
QUELLI CHE VEDREMO



Ovvero il 20% di UML che vi aiuta a fare 80% del lavoro!

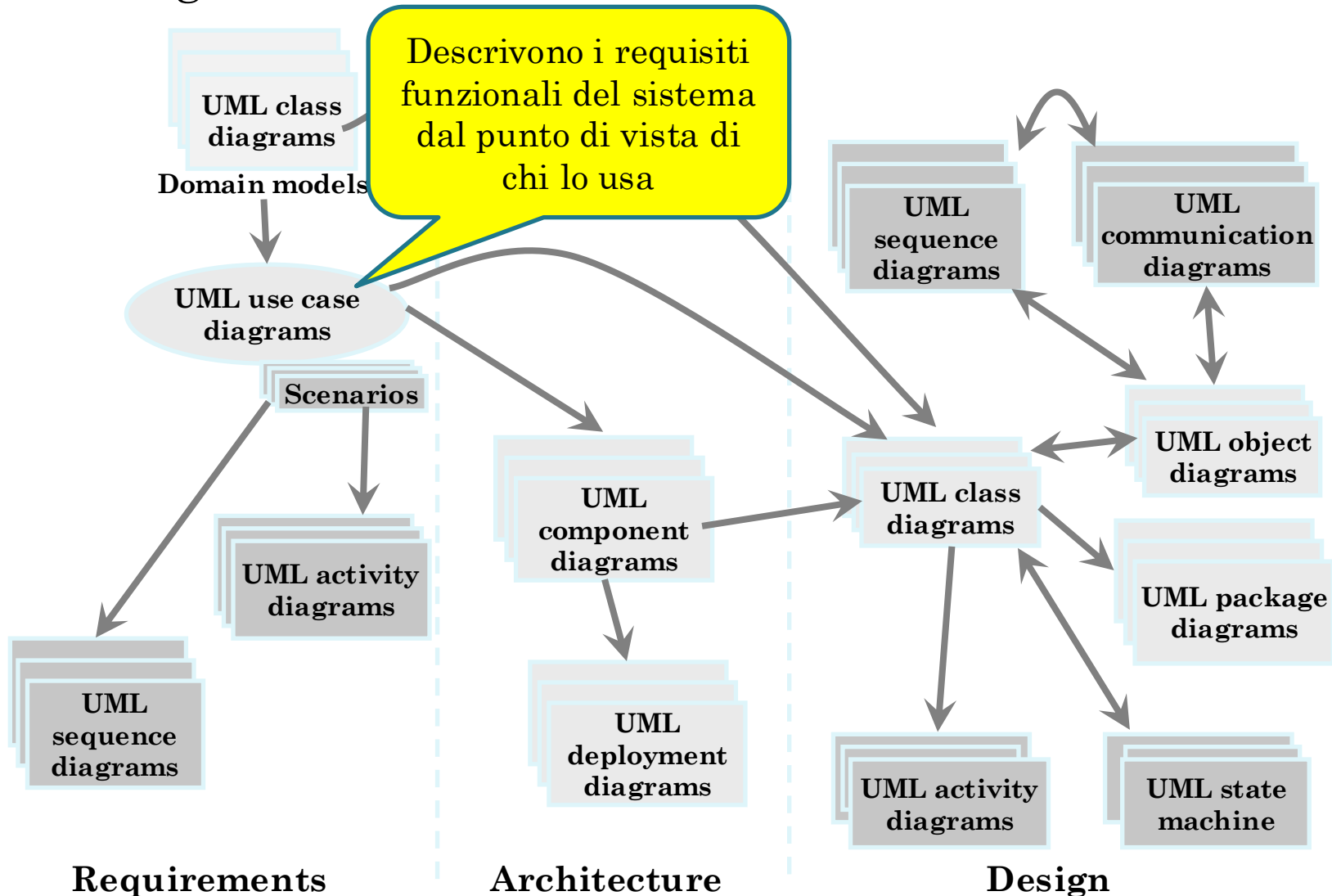
INTEGRARE UML NEL PROCESSO

- UML può esser usato con qualsiasi processo nelle seguenti fasi:



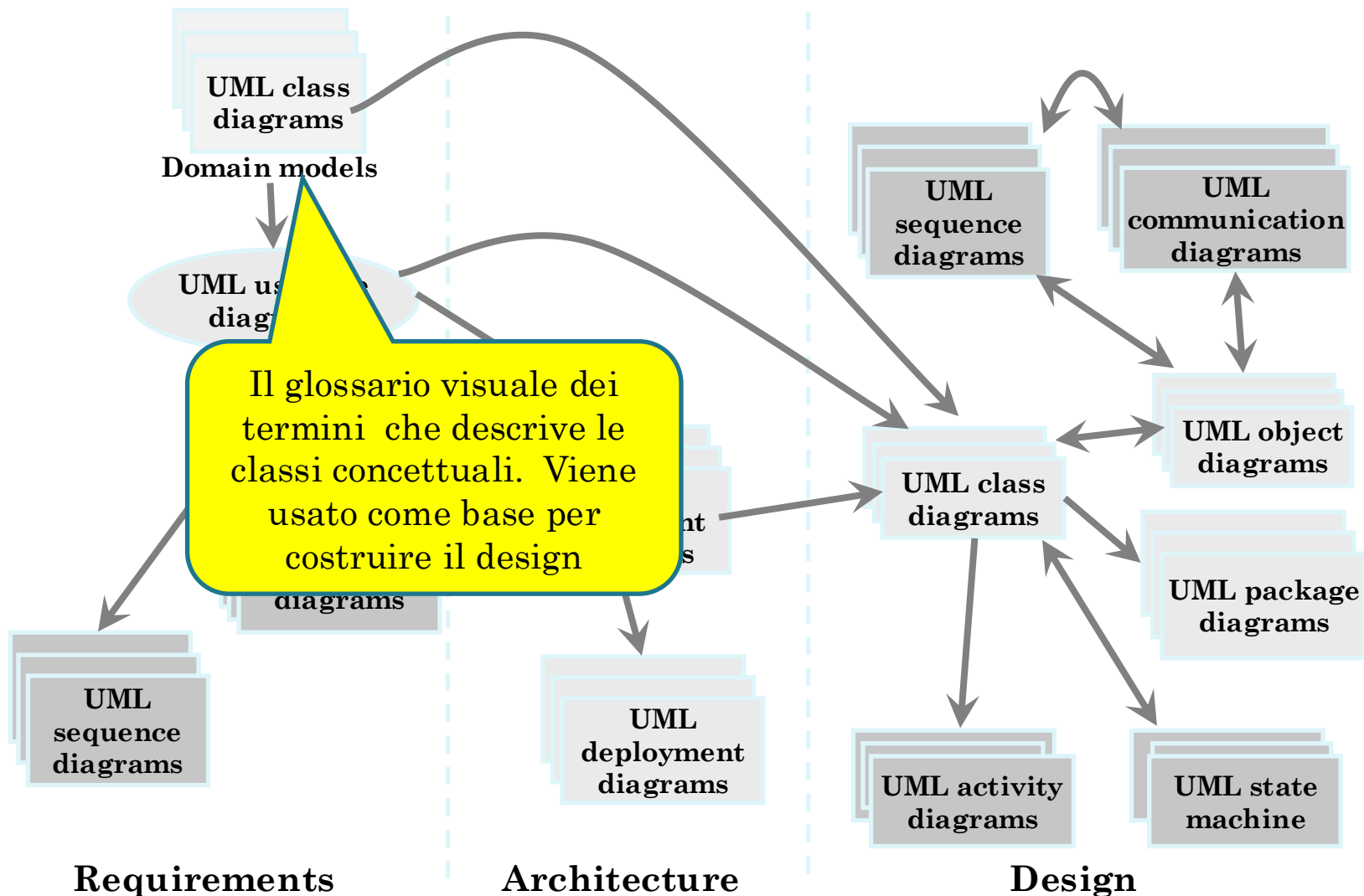
INTEGRARE UML NEL PROCESSO

- UML può esser usato con qualsiasi processo nelle seguenti fasi:



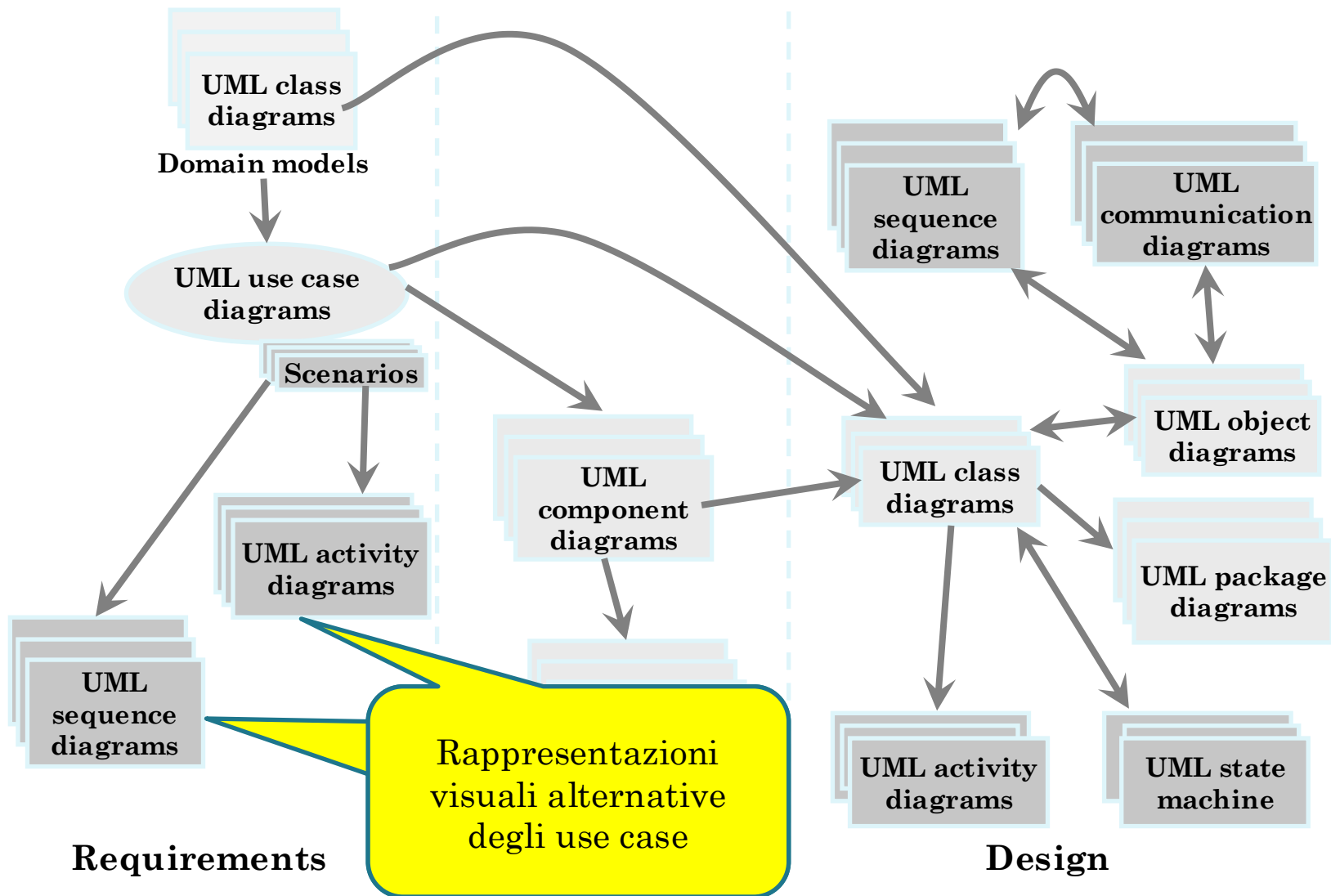
INTEGRARE UML NEL PROCESSO

- UML può esser usato con qualsiasi processo nelle seguenti fasi:



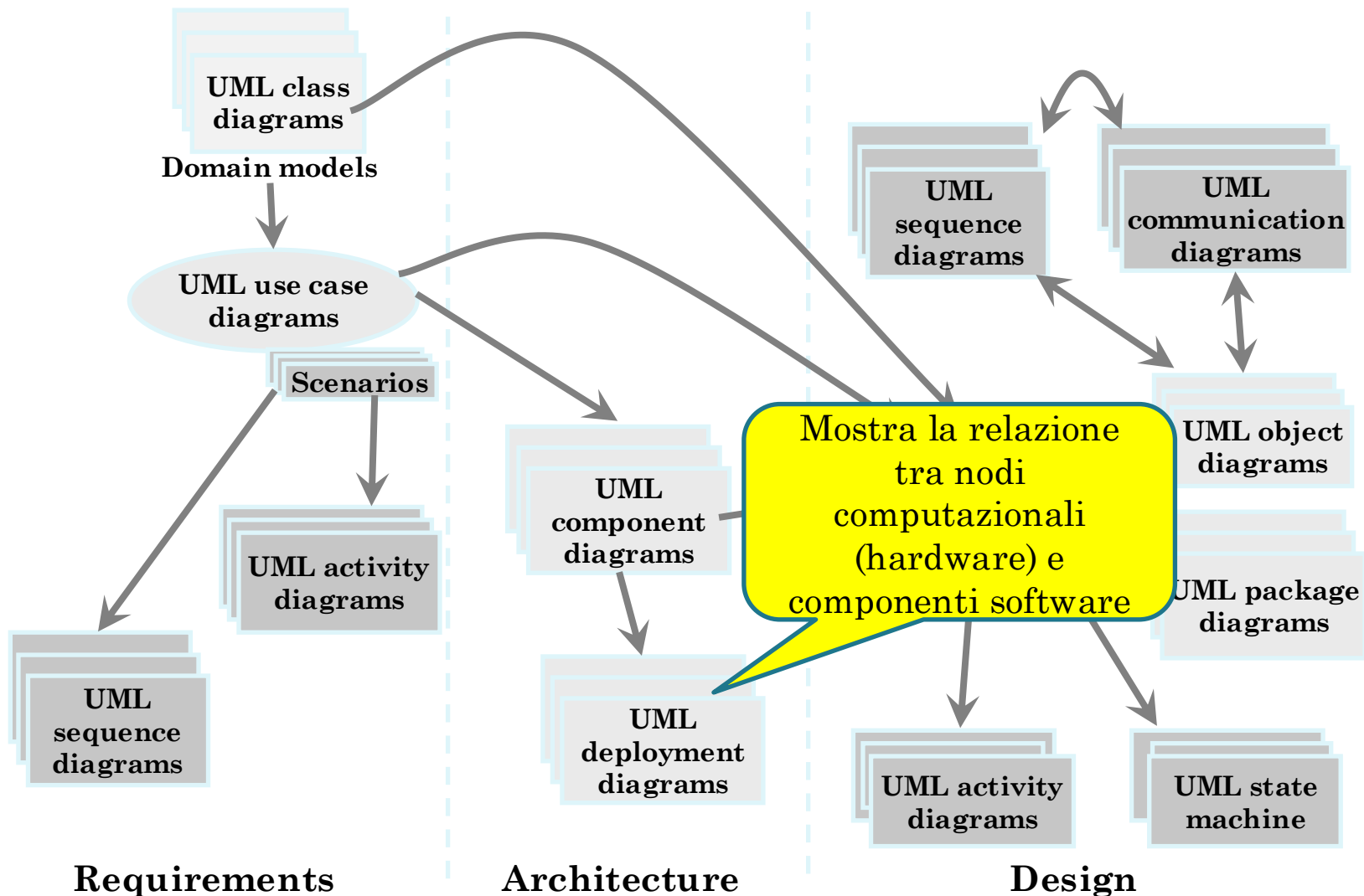
INTEGRARE UML NEL PROCESSO

- UML può esser usato con qualsiasi processo nelle seguenti fasi:



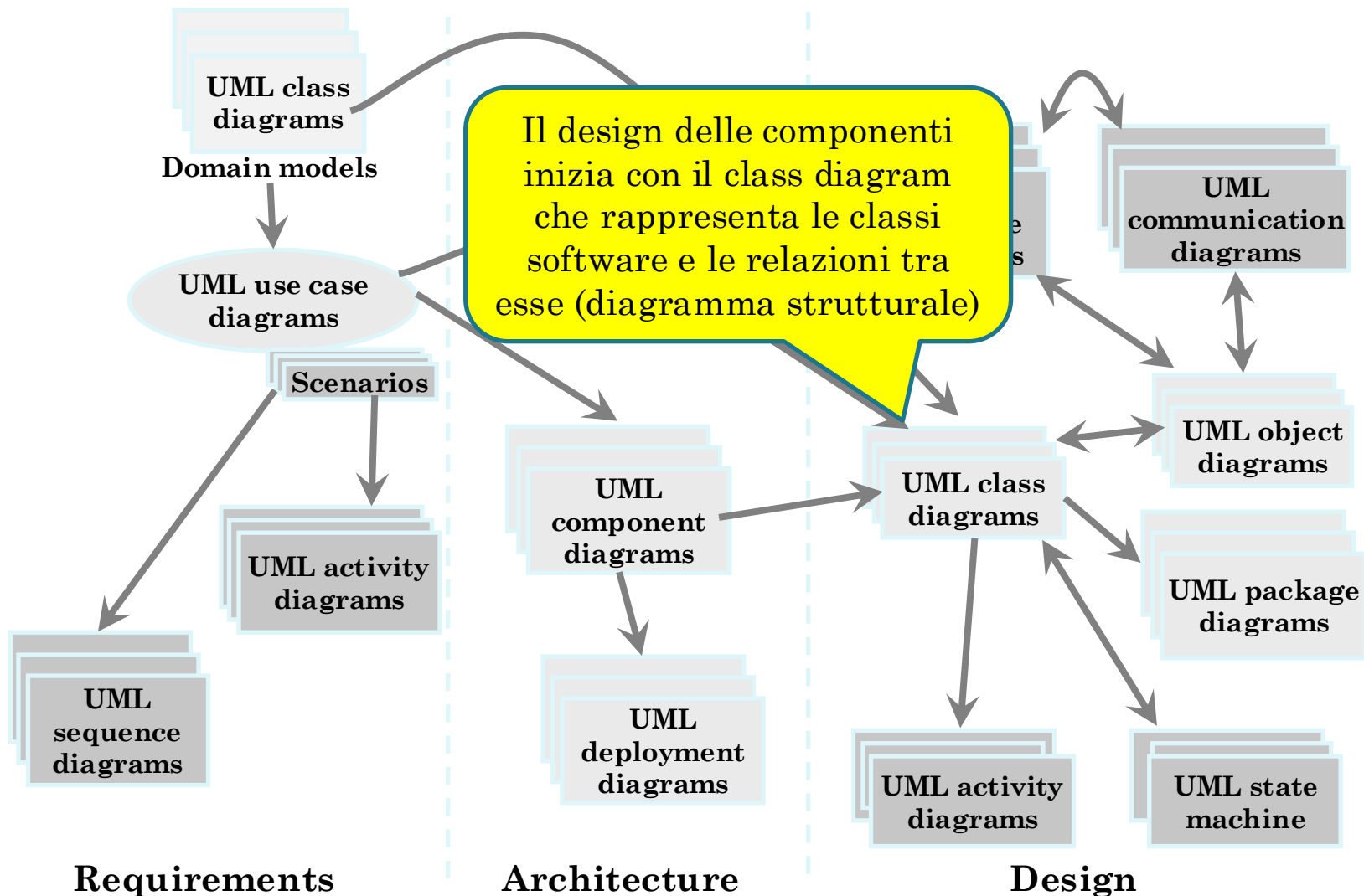
INTEGRARE UML NEL PROCESSO

- UML può esser usato con qualsiasi processo nelle seguenti fasi:



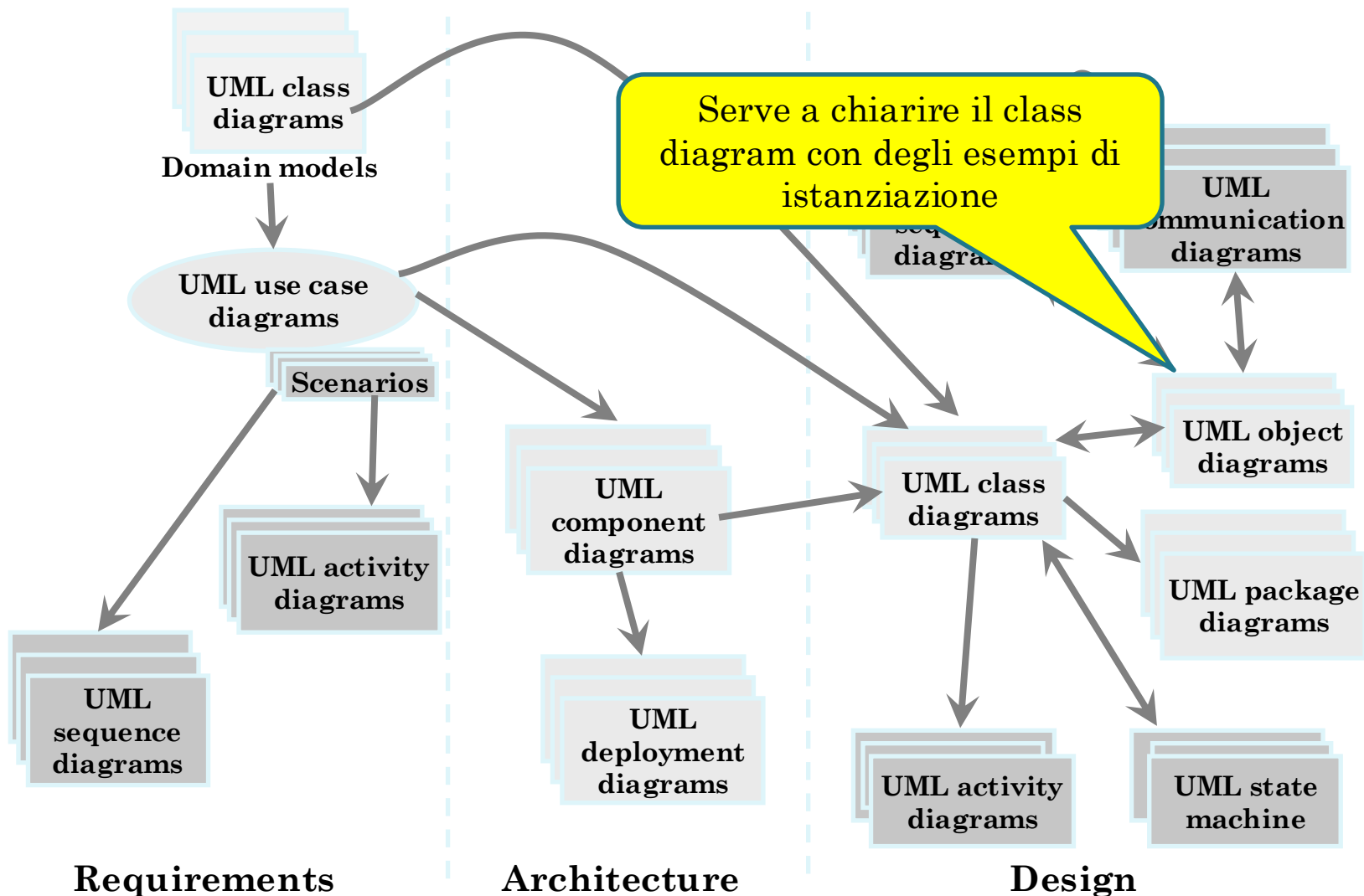
INTEGRARE UML NEL PROCESSO

- UML può esser usato con qualsiasi processo nelle seguenti fasi:



INTEGRARE UML NEL PROCESSO

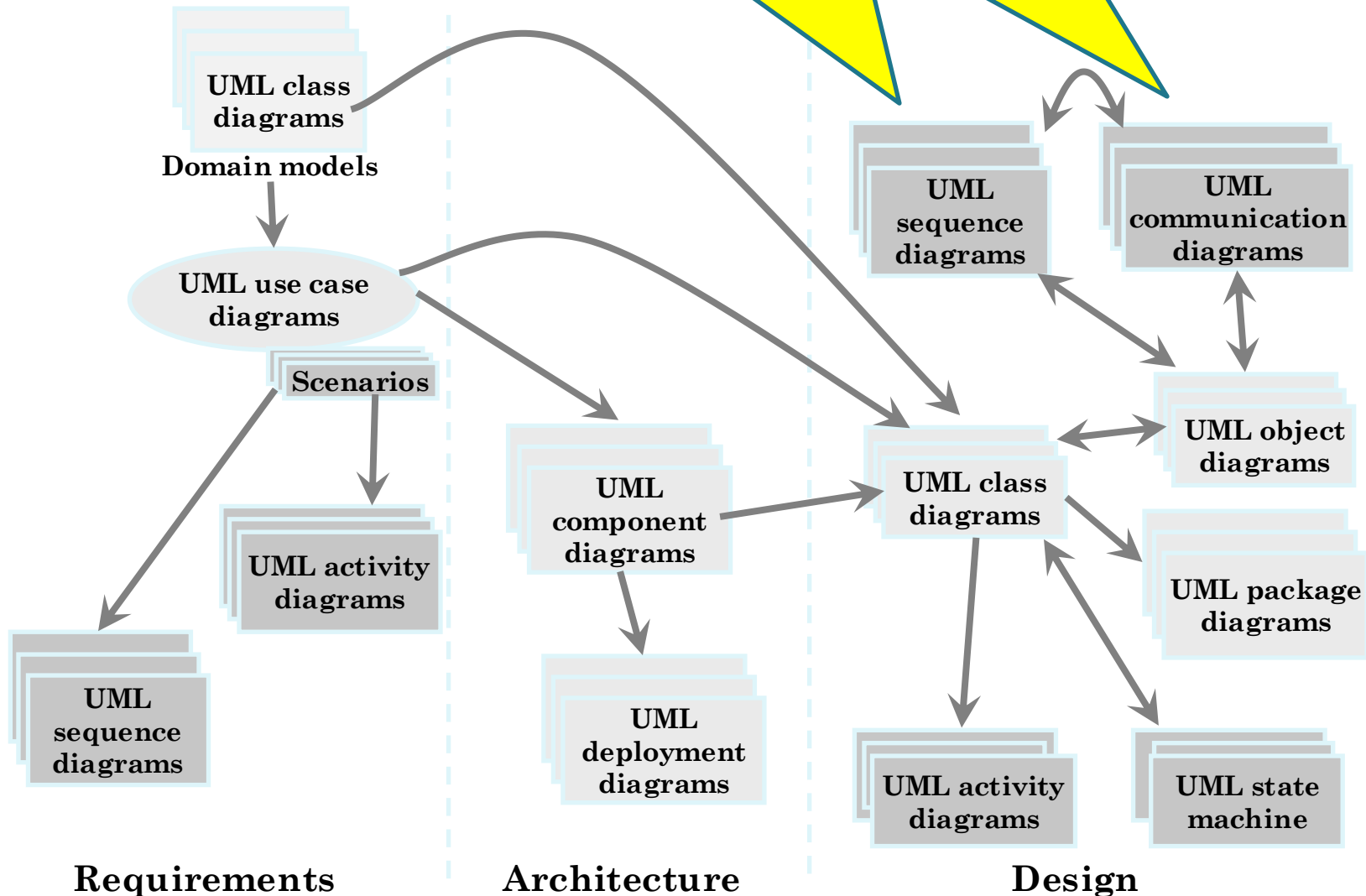
- UML può esser usato con qualsiasi processo nelle seguenti fasi:



INTEGRARE UML NEL PROCESSO

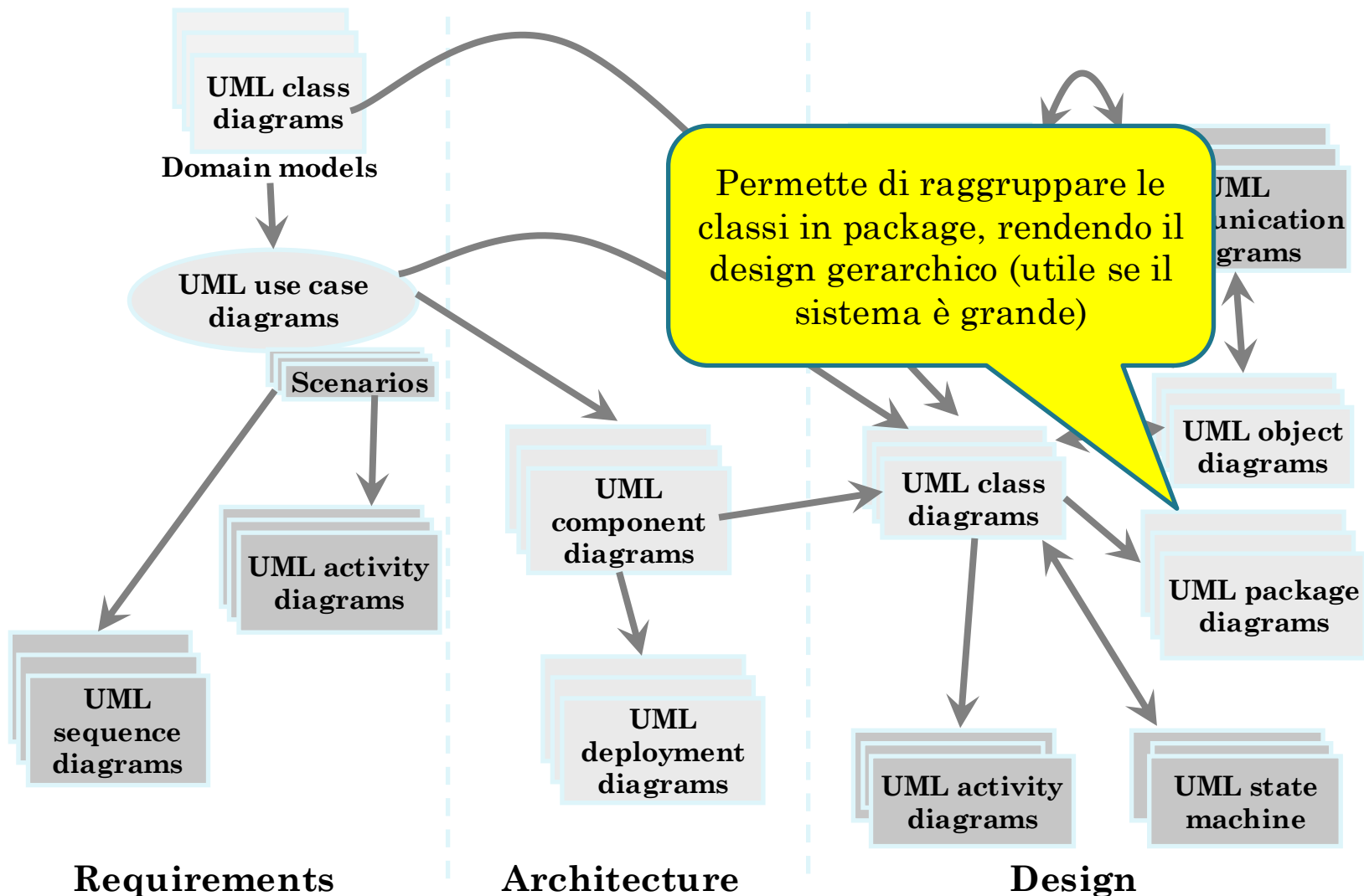
- UML può esser usato nelle seguenti fasi:

Mostrano come i messaggi fluiscono da un oggetto ad un altro



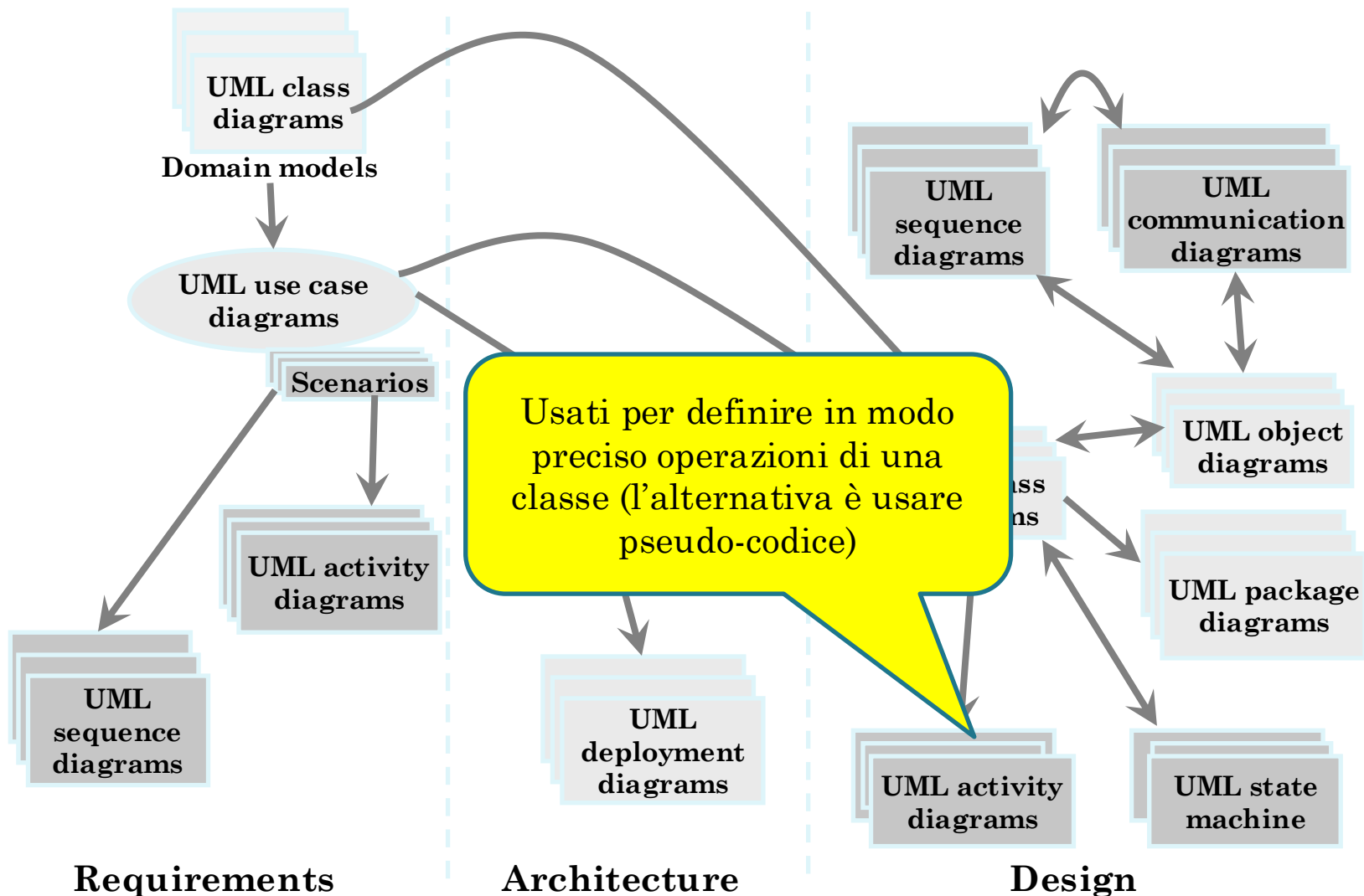
INTEGRARE UML NEL PROCESSO

- UML può esser usato con qualsiasi processo nelle seguenti fasi:



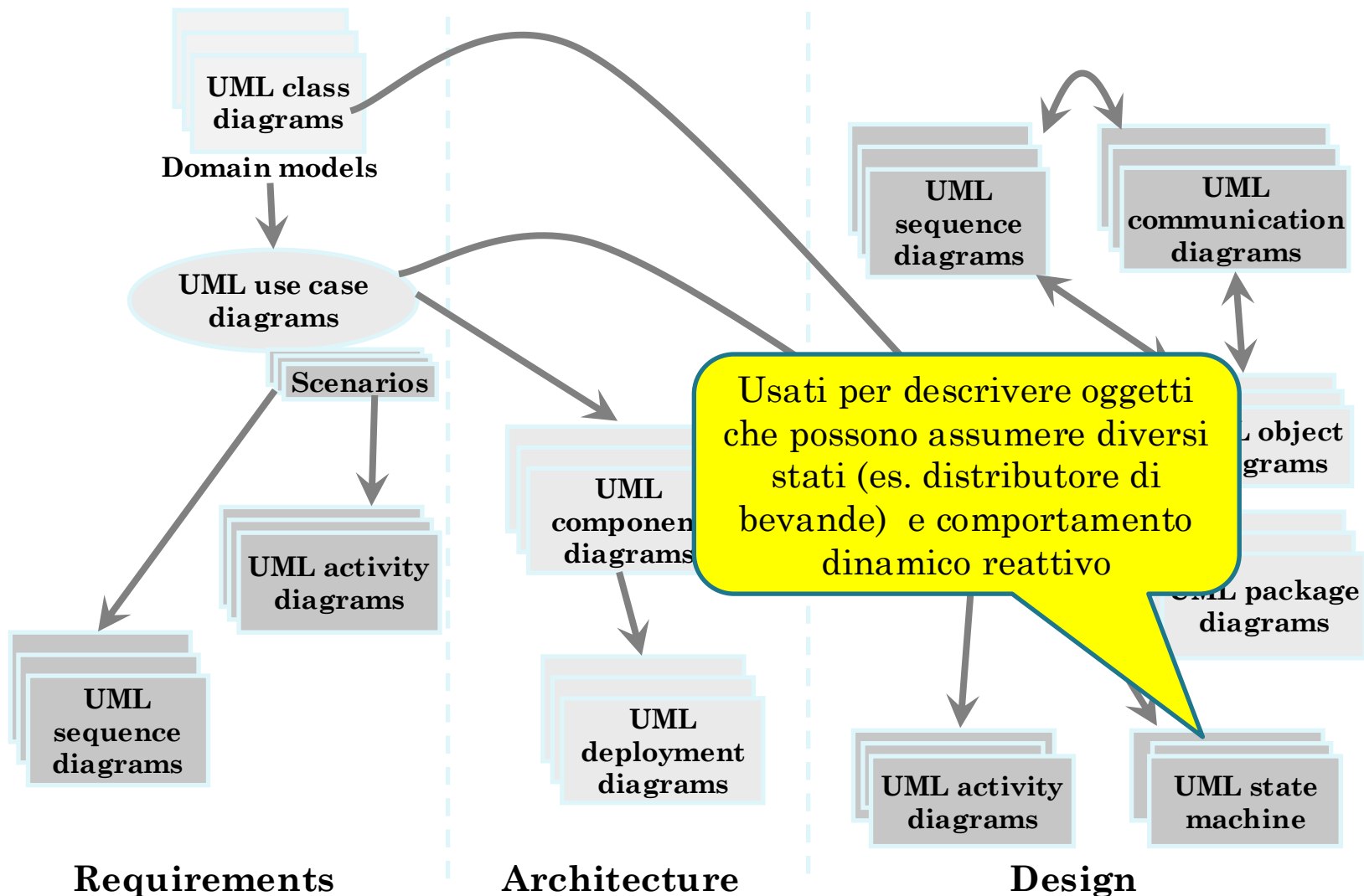
INTEGRARE UML NEL PROCESSO

- UML può esser usato con qualsiasi processo nelle seguenti fasi:



INTEGRARE UML NEL PROCESSO

- UML può esser usato con qualsiasi processo nelle seguenti fasi:



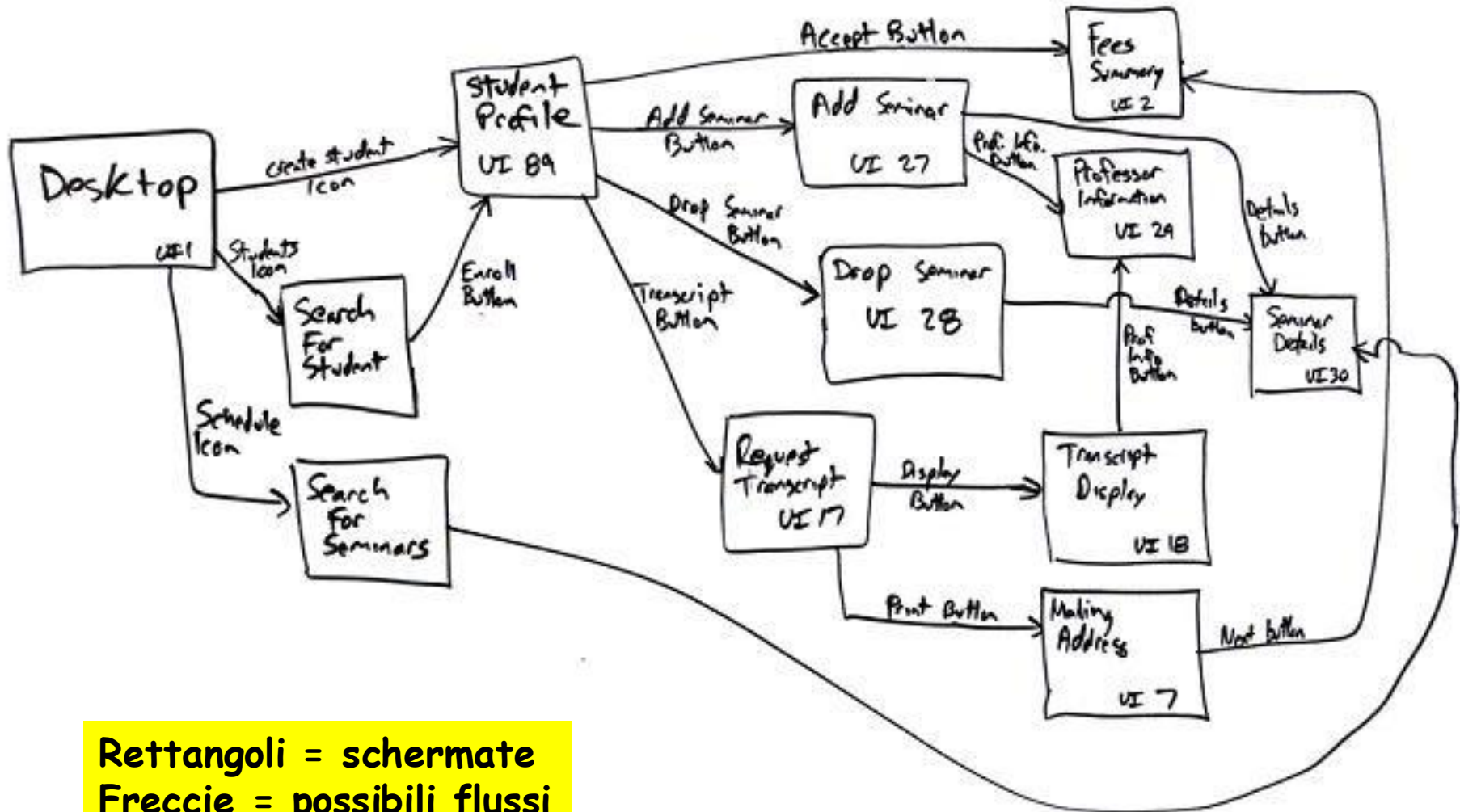
UML NON È ABBASTANZA

- Benché i diagrammi UML siano tanti l'insieme non è esaustivo ...
- Esistono due soluzioni a questo problema:
 - Si cerca o si produce un “estensione” di UML
 - Vedi il concetto di profilo (+ avanti)
 - Si usano diagrammi non UML
 - M. Fowler: *“non esitare a farlo se UML non ne comprende uno adatto al vostro scopo”*

Ma si perde uno dei vantaggi di UML, la standardizzazione

Molto utile nelle applicazioni Web

DIAGRAMMA DELLE SCHERMATE (STORYBOARD)



PROFILO

- UML si può **estendere** tramite il concetto di **profilo** per avere la possibilità di specificare particolari **domini applicativi** o **tipologie di applicazioni**
 - Es.
 - **Dominio applicativo:** dominio ferroviario, spaziale
 - **Tipologia:** applicazioni WEB o sistemi embedded
- Un **profilo** è costituito principalmente da:
 - **Stereotipi**, ovvero elementi aggiuntivi ottenuti modificando quelli esistenti
 - **Vincoli aggiuntivi** (Constraint) rispetto a quelli standard presenti in UML
 - per *vietare* l'uso di alcuni elementi del linguaggio
 - **Informazioni semantiche aggiuntive** relative agli elementi aggiuntivi

PROFILO PER APPLICAZIONI WEB (CONALLEN)

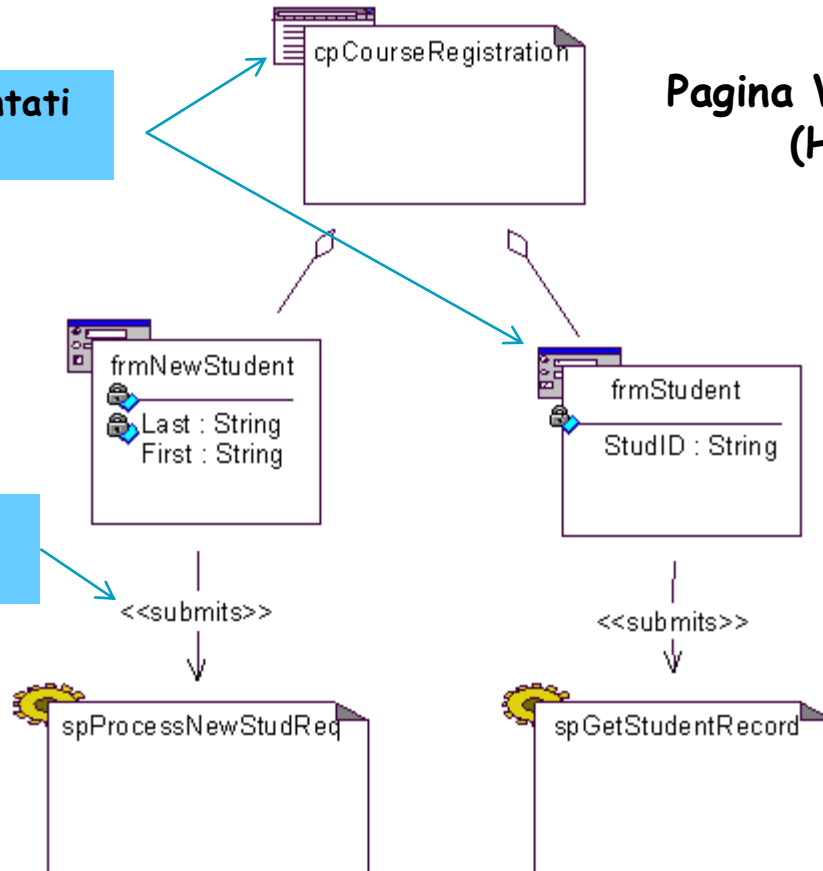
Stereotipi rappresentati
da icone nuove

Pagina Web statica
(HTML)

Altra possibilità per rappresentare
gli stereotipi è usare << ... >>

Form

Server Page



UML E IL CODICE ...

- **Aspetto fastidioso:** non esiste una definizione della corrispondenza tra UML e un qualsiasi linguaggio di programmazione ...
- Non si può guardare un diagramma UML e dire esattamente cosa farà il codice corrispondente ...
 - Ma solo avere una buona approssimazione di cosa farà ...
- Modelli UML sono ad un livello di astrazione più alto!!
 - Unica eccezione se usiamo UML come linguaggio di programmazione



QUALE UML È LEGALE?

- Quello definito dalla specifica del linguaggio
 - Ovvero dal documento OMG
 - <http://www.omg.org/>, cercare UML
- Ma UML è molto complesso e si presta a multiple interpretazioni
- Così le persone lo usano adottando **convenzioni particolari**
- **“Uno, due molti UML ...” [Fowler]**
 - normativo/standard/legale rispetta lo standard
 - UML convenzionale che non rispetta lo standard ma di ampio uso
 - Di solito si aggiunge nei diagrammi **“non-normativo”**

QUALE UML È LEGALE?

- Quello definito dalla specifica del linguaggio
 - Ovvero dal documento OMG
 - <http://www.omg.org/>, cercare UML
- Ma UML è molto complesso e si presta a multiple

Dato che lo scopo primario dei diagrammi UML è quello di comunicare è meglio capirsi che rispettare lo standard!

particolari

- **“Uno, due molti UML ...” [Fowler]**
 - normativo/standard/legale rispetta lo standard
 - UML convenzionale che non rispetta lo standard ma di ampio uso
 - Di solito si aggiunge nei diagrammi **“non-normativo”**

CONSIGLI SU COME USARE UML

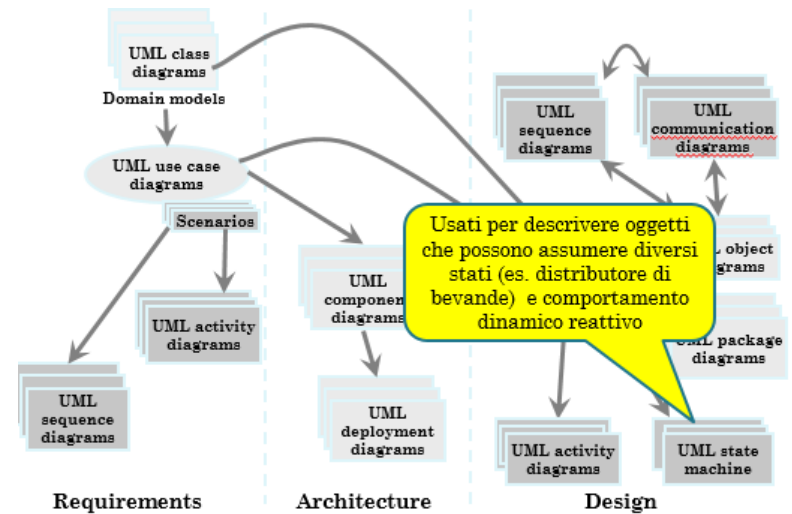
1. Capire il punto di vista dell'autore per capire i diagrammi
 - Utilizzo/Prospettiva
 - Es. Abbozzo/Software
2. Attenzione: **qualsiasi info può essere sempre soppressa**
 - l'assenza di qualcosa non dimostra nulla!
 - Es. X: int senza inizializzazione, il valore esiste ma è stato omesso oppure non esiste e quindi si inizializza con il valore di default?
3. Meglio dei buoni diagrammi **“illegali”** che altri formalmente corretti ma scarsi
4. UML è molto complesso da prestarsi a multiple interpretazioni: usate un sottoinsieme e solo i diagrammi che ritenete utili (e che avete compreso bene)
5. La completezza è nemica della chiarezza

The logo for the Unified Modeling Language (UML). It features the text "UNIFIED MODELING LANGUAGE" in a dark blue, sans-serif font, stacked vertically. To the right of the text is a large, stylized, 3D block letter "UML". The "U" is maroon, the "M" is yellow, and the "L" is dark blue. A small "TM" trademark symbol is located to the right of the "L".

```

graph TD
    Diagram[Diagram] --> Structure[Structure Diagram]
    Diagram --> Behavior[Behavior Diagram]
    Structure --> Class[Class Diagram]
    Structure --> Component[Component Diagram]
    Structure --> Object[Object Diagram]
    Structure --> Composite[Composite Structure Diagram]
    Structure --> Deployment[Deployment Diagram]
    Structure --> Package[Package Diagram]
    Behavior --> Activity[Activity Diagram]
    Behavior --> UseCase[Use Case Diagram]
    Behavior --> StateMachine[State Machine Diagram]
    Behavior --> Interaction[Interaction Diagram]
    Interaction --> Sequence[Sequence Diagram]
    Interaction --> Overview[Interaction Overview Diagram]
    Interaction --> Communication[Communication Diagram]
    Interaction --> Timing[Timing Diagram]
  
```

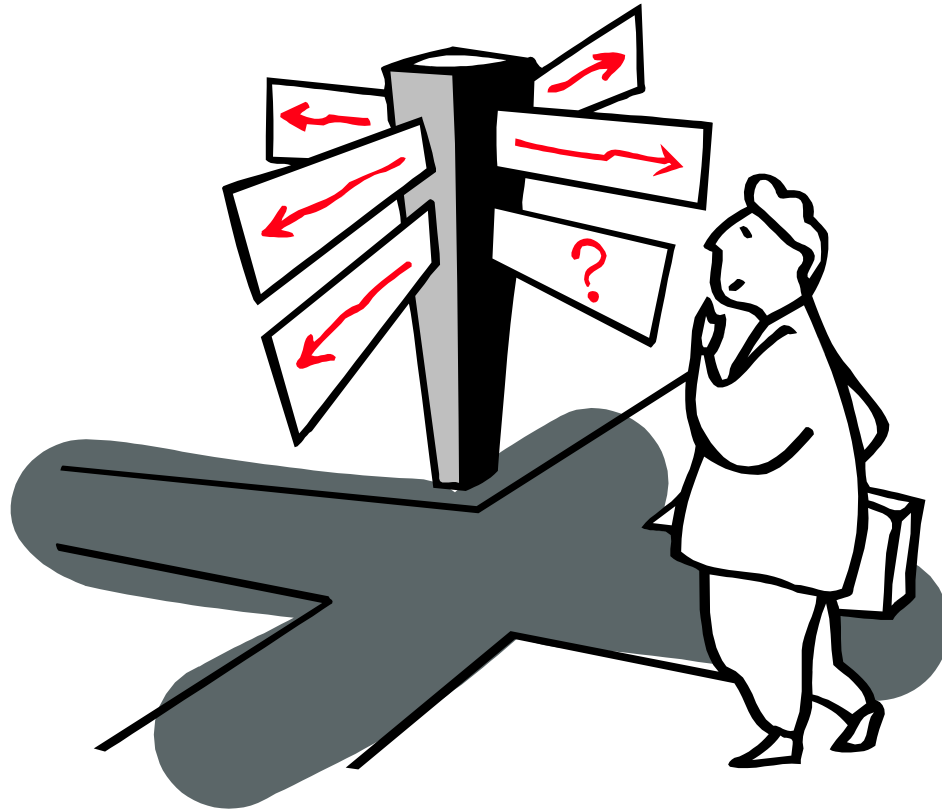
The diagram illustrates the hierarchy of UML diagrams. At the top is the **Diagram** box. It branches into **Structure Diagram** and **Behavior Diagram**. **Structure Diagram** further branches into **Class Diagram**, **Component Diagram**, **Object Diagram**, **Composite Structure Diagram**, **Deployment Diagram**, and **Package Diagram**. **Behavior Diagram** branches into **Activity Diagram**, **Use Case Diagram**, **State Machine Diagram**, and **Interaction Diagram**. **Interaction Diagram** further branches into **Sequence Diagram**, **Interaction Overview Diagram**, **Communication Diagram**, and **Timing Diagram**. Arrows point from the child boxes to their parent boxes, indicating inheritance or specialization.



INFO SU UML

- Definizioni ufficiali (<http://www.omg.org/>)
 - Unified Modeling Language: Superstructure
 - Il documento di riferimento
 - sintassi e semantica
- J. Rumbaugh, G. Booch and I. Jacobson. *The Unified Modeling Language Reference Manual* (piuttosto vecchio)
- M. Fowler, *UML Distilled: A Brief Guide to the Standard Object Modeling Language*
- R. S. Pressman, *Principi di Ingegneria del software* 5/ed,
 - Capitolo 7 relativo ad UML (a cura di Cerioli e Reggio)
- **Attenzione al materiale che si trova su Web**
 - **attenzione anche a cose che si riferiscono a UML 1**

THE END ...



Domande?